INTERACTIVE BUSINESS INTELLIGENCE DASHBOARD

Project Report

Project type	Interactive BI dashboard
Framework	Python + Streamlit
Visualization	Plotly
Dataset	Synthetic regional sales sample
Coverage	January–June 2026
Prepared for	Academic / internship project submission

Data transparency note: The included dataset is synthetic demonstration data, not actual business data. Replace it with an approved real dataset before reporting business conclusions.

Contents

1. Abstract

2. Introduction and Problem Statement

3. Objectives and Scope

4. Tools and Technologies

5. Dataset and Data Preparation

6. Dashboard Design and KPI Definitions

7. Implementation and Workflow

8. Testing and Validation

9. Results and Insights

10. Deployment Instructions

11. Limitations and Future Scope

12. Conclusion

13. References

1. Abstract

This project develops an interactive Business Intelligence (BI) dashboard that summarizes business performance through key performance indicators (KPIs), charts, and filters. The application is built with Python, Streamlit, Pandas, and Plotly. Users can filter records by date range, region, and product category, then explore revenue trends, regional performance, category mix, active users, and churn. The dashboard also allows users to inspect and download the filtered records. A small synthetic dataset is supplied so the project can run immediately without access to confidential business information.

The purpose is to demonstrate the end-to-end dashboard workflow: importing structured data, checking its quality, calculating metrics, visualizing patterns, and preparing a deployable application. Because the sample data is generated for demonstration, any apparent trends should not be treated as real market findings.

2. Introduction and Problem Statement

Organizations often store sales and customer information in spreadsheets or databases. Reviewing many rows manually makes it harder to spot changes, compare regions, and monitor customer activity. A dashboard brings the most useful measures into one interface and lets a user narrow the analysis to a specific time period or business segment.

Problem statement: create a simple, interactive, and accessible dashboard that turns structured regional sales data into readable KPIs and charts, while allowing users to explore the data without editing code.

3. Objectives and Scope

• Import and validate a structured CSV dataset.

• Display Total Revenue, Active Users, Churn Rate, and an average-ticket-size proxy.

• Provide dynamic date-range, region, and category filters.

• Present responsive visualizations for monthly revenue, regional revenue, category mix, and active users.

• Show filtered rows and allow CSV export.

• Document local execution and public deployment steps.

Scope: this is a demonstration dashboard for learning and academic submission. It does not include login, role-based access, a production database, automated data refresh, or a connection to a live business system.

4. Tools and Technologies

Technology	Purpose
Python	Application logic and calculations
Pandas	CSV import, data cleaning, grouping, aggregation
Streamlit	Web interface, filters, KPI cards, layout
Plotly Express	Interactive line, bar, and donut charts
Jupyter Notebook	Data checks and summary analysis
CSV	Portable sample dataset format
GitHub + Streamlit Community Cloud	Optional source hosting and public deployment

5. Dataset and Data Preparation

The provided CSV contains 72 demonstration records across six monthly dates (January–June 2026), four regions (Central, North, South, West), and three categories (Electronics, Home & Living, Fashion). Each record contains Date, Region, Category, Revenue, Active_Users, and Churned_Customers. Values are synthetic and created to support the dashboard demonstration.

Data preparation steps

• Read the CSV file with Pandas and parse Date as a date/time field.

• Convert Revenue to numeric values and fill invalid revenue entries with zero in the app.

• Convert Active_Users and Churned_Customers to integer values, filling invalid entries with zero.

• Remove records missing essential Date, Region, or Category values.

• Apply the selected date, region, and category filters before calculating KPIs and charts.

In a real project, missing values should be investigated rather than automatically replaced without review. The data owner should confirm units, duplicates, date definitions, and metric definitions before the dashboard is used for decisions.

6. Dashboard Design and KPI Definitions

KPI cards

KPI	Definition used in this demo
Total Revenue	Sum of Revenue for the filtered records.
Active Users	Sum of Active_Users for filtered records; may count the same user more than once across rows.
Churn Rate	Sum of Churned_Customers ÷ sum of Active_Users × 100. This is a simplified demo definition.
Avg Ticket Size (proxy)	Revenue ÷ Active_Users because the sample has no order-count field. It is not a true average order value

Filters and charts

• Date range: limits the selected reporting period.

• Region: compares selected geographic areas.

• Category: focuses analysis on selected product groups.

• Monthly Revenue Trend: line chart to examine changes over time.

• Revenue by Region: bar chart to compare regional revenue.

• Revenue Mix by Category: donut chart showing each category’s contribution.

• Active Users by Region: bar chart comparing user totals by region.

• Filtered Data: interactive table and CSV download for further analysis.

7. Implementation and Workflow

The application loads the CSV from the same folder as app.py. Streamlit caches the data-loading function to avoid repeated reads during ordinary interactions. Sidebar controls define the selected dates, regions, and categories. Pandas filters the data, computes KPI totals, and groups records for charting. Plotly creates interactive charts, while Streamlit renders the charts and data table in a browser-based interface.

Workflow: CSV Dataset → Load and Validate → Apply Filters → Calculate KPIs → Create Charts → Display Table → Export Filtered CSV

Run locally

```
pip install -r requirements.txt
streamlit run app.py
```

After the command starts, open the local address shown in the terminal, usually http://localhost:8501. Keep app.py and dashboard_sample_dataset.csv in the same folder.

8. Testing and Validation

Test	Expected result
Application start	Streamlit launches without missing-package errors after dependencies are installed.
CSV loading	Dataset loads and Date is parsed as a date field.
Date filter	Charts and KPIs update to the selected date range.
Region filter	Only selected regions contribute to the displayed values.
Category filter	Only selected categories contribute to the displayed values.
No matching records	A warning is displayed rather than misleading charts.
CSV download	Filtered rows download as a CSV file.
Data transparency	Dashboard clearly labels the dataset as synthetic.

These are the planned validation cases. Execute them on the target computer and record any environment-specific issues before final submission.

9. Results and Insights

The completed application is designed to show the total revenue, active-user count, churn rate, and average-ticket-size proxy for the current filters. Four charts provide complementary views of the data: trend over time, comparison across regions, category contribution, and user totals by region. The filtered data table and CSV download support additional inspection.

No real business conclusions are claimed because all included data is synthetic. After replacing the sample with authorized data, the analyst can investigate which regions contribute most to revenue, whether category shares change over time, and whether churn differs across segments. Findings should be checked against the organization’s official definitions and source systems.

10. Deployment Instructions

• Create a GitHub repository and upload app.py, requirements.txt, dashboard_sample_dataset.csv, and README.md.

• Open https://share.streamlit.io/ and sign in using GitHub.

• Create a new app, select the repository and branch, and enter app.py as the main file path.

• Deploy and wait for the service to install dependencies and start the app.

• Open the public URL, test all filters and charts, and include the working URL in the submission.

A live public URL is not embedded in this report because deployment must be completed using the student’s own account. If publishing is not possible, record a short walkthrough video showing the app, KPI cards, filters, charts,

and CSV download; follow the instructor’s required submission format.

11. Limitations and Future Scope

Limitations

• Synthetic dataset only; it does not represent actual sales or customers.

• No order count, so average ticket size is only a proxy.

• Active-user totals may double-count users across records.

• Churn formula is simplified and may differ from standard business definitions.

• No database connection, user login, access control, or automated refresh.

• Public deployment requires a separate account and internet access.

Future scope

• Connect to a database or secured cloud data source.

• Add an Orders field and calculate true average order value.

• Use unique customer IDs to calculate distinct active users.

• Define churn by cohort and a documented time window.

• Add targets, prior-period comparisons, and alerts.

• Add authentication, role-based access, and scheduled refresh for production use.

• Improve mobile layout and add a downloadable PDF snapshot.

12. Conclusion

This project demonstrates how structured data can be converted into an interactive dashboard using Python, Pandas, Streamlit, and Plotly. It provides core KPI cards, dynamic filters, multiple visualizations, a data table, and CSV export. The included files support local execution and optional public deployment. Before practical business use, replace the synthetic dataset, verify the metric definitions, and test the application with the intended audience.

13. References

• Streamlit documentation: https://docs.streamlit.io/

• Pandas documentation: https://pandas.pydata.org/docs/

• Plotly Python documentation: https://plotly.com/python/

• Streamlit Community Cloud: https://docs.streamlit.io/deploy/streamlit-community-cloud

• Python documentation: https://docs.python.org/3/